gymbro

Setup Manual

A step-by-step guide to running the React Native workout tracker on your own device.

Repository:

https://github.com/gagangeet2517-arch/gymbro

Built with Expo, React Native, and Expo Router

1. What is gymbro?

gymbro is a workout-tracking mobile app written with React Native, Expo, and Expo Router. It runs on iOS, Android, and the web from a single codebase. Your workout, nutrition, and body-metric data is stored locally on-device using AsyncStorage — there is no backend and no account system.

Core tracking (workouts, templates, history, progress charts) works fully offline. The optional AI features — food-photo scanning, voice meal logging, and the nutrition coach — call Google Gemini, so they need an internet connection and a Gemini API key (see section 6). Everything else runs without a network.

This manual walks through every step required to clone the project, install its dependencies, run it on a simulator, emulator, or physical device, and enable the optional AI features.

2. Prerequisites

Before you start, make sure the following are installed on your machine.

2.1 Node.js (v20.19 or newer)

Expo SDK 54 requires Node 20.19+ or Node 22+. Verify your version:

```
node --version
```

If it is missing or too old, download the LTS installer from nodejs.org, or use a version manager such as nvm or fnm:

```
# Using nvm
nvm install 22
nvm use 22
```

2.2 Git

Required for cloning the repository.

```
git --version
```

On macOS, install via Xcode Command Line Tools:

```
xcode-select --install
```

On Windows, download from git-scm.com. On Linux, use your package manager (e.g. sudo apt install git).

2.3 A package manager

npm ships with Node, so nothing extra to install. If you prefer yarn or pnpm, they will work too — the examples in this guide use npm.

2.4 Platform tooling (pick at least one)

You need a way to actually see the app running. Pick whichever option fits your operating system and target platform:

- **iOS Simulator (macOS only)** — install Xcode from the Mac App Store, then open it once to accept the license. After that run `xcode-select --install` if you have not already.
- **Android Emulator (any OS)** — install Android Studio, open it, and use the AVD Manager to create a virtual device (Pixel 7, API 34 is a safe default). Make sure ANDROID_HOME points to your SDK folder.
- **Physical phone with Expo Go** — the fastest way to test on a real device. Install the free "Expo Go" app and scan the QR code Expo prints when you start the dev server. Phone and computer must share a Wi-Fi network.
- **Physical iPhone, native build** — to install a full standalone build on your own iPhone (required for camera-based food scanning), see section 7.
- **Web browser** — no extra tools needed; some native features fall back gracefully.

> **Note:** On macOS the recommended combo is the iOS Simulator (built in) plus Expo Go on your phone. On Windows or Linux, use the Android Emulator and/or Expo Go.

3. Cloning the repository

Open a terminal in the folder where you want the project to live, then run:

```
git clone https://github.com/gagangeet2517-arch/gymbro.git
cd gymbro
```

If you already have a local copy and just want to pull the latest changes:

```
cd gymbro
git pull origin main
```

4. Installing dependencies

From the project root, install everything listed in package.json:

```
npm install
```

This pulls down React, React Native, Expo, the navigation libraries, AsyncStorage, react-native-svg (used for the progress charts), and the rest of the dependencies. The first install can take a few minutes.

> **Note:** If the install fails with a peer-dependency error, try `npm install --legacy-peer-deps`. If it fails with a permissions error, never use sudo — fix the npm prefix instead (see troubleshooting).

5. Running the app

All run commands start with `npx expo` so you do not need to install the Expo CLI globally.

5.1 Start the dev server

```
npx expo start
```

This launches the Metro bundler and shows a menu in your terminal along with a QR code. From here you can press a key to open the app:

- Press **i** to open the iOS Simulator (macOS).
- Press **a** to open the Android Emulator (or a connected device).
- Press **w** to open the web build in your default browser.
- Or scan the QR code with your phone (Camera app on iOS, the Expo Go app on Android).

5.2 Open a specific platform directly

```
npx expo start --ios
npx expo start --android
npx expo start --web
```

5.3 Reload after code changes

Saving a file triggers Fast Refresh automatically. If state gets stuck, press **r** in the dev-server terminal to force a full reload, or shake the device to open the developer menu.

6. Enabling AI features (optional)

Food-photo scanning, voice meal logging, and the nutrition coach run on Google Gemini. They are entirely optional — the rest of the app works without them — but to use them you need a free Gemini API key.

6.1 Get a free key

Sign in at aistudio.google.com/apikey and create an API key. It is free for the usage limits this app needs and takes about a minute.

6.2 Add the key in the app

Open the app, go to the **Home** tab and open your **Profile**. Under "AI features · Gemini key", paste your key and tap Save profile. The key is stored locally on the device and is used in preference to any shared key.

6.3 Optional environment fallback (developers)

For local development you can instead provide keys via a `.env.local` file at the project root. A key entered in Profile always takes priority over these:

```
EXPO_PUBLIC_GOOGLE_AI_KEY=your_key_here
# optional extra keys, tried in order if the first is rate-limited
EXPO_PUBLIC_GOOGLE_AI_KEY_2=...
EXPO_PUBLIC_GOOGLE_AI_KEY_3=...
```

> **Note:** Never commit real API keys. `.env.local` is git-ignored; the in-app Profile key never leaves the device.

7. Installing on a physical iPhone

To run a full native build on your own iPhone (needed for the camera-based food scanner), build and install it over USB. A convenience script is wired up in package.json:

```
npm run deploy:iphone
```

This runs a Release xcodebuild and installs the resulting .app onto the connected device with `xcrun devicectl`. Before the first run:

- Connect the iPhone over USB and trust the computer.
- Open `ios/gymbro.xcworkspace` in Xcode once, sign in with your Apple ID under Signing & Capabilities, and pick your team so provisioning is set up.
- Update the device id and DEVELOPMENT_TEAM in the deploy:iphone script to match your device and Apple ID (find the device id with `xcrun devicectl list devices`).

> **Note:** Builds signed with a free (non-paid) Apple ID expire after 7 days. When the app stops launching, plug the phone back in and re-run `npm run deploy:iphone` to refresh it. Reinstalling keeps your existing on-device data.

8. Verifying your setup

Once the app loads you should see the Home tab. To confirm everything is wired up:

- Navigate between the bottom tabs — none should crash.
- Open Workouts — starter templates are seeded automatically on first launch.
- Start a workout, log a set, and finish it. Check the History tab to confirm it saved.
- Force-quit and reopen the app. Your data should persist because it lives in AsyncStorage.
- (If you added a Gemini key) open Nutrition and try the photo scan to confirm AI features work.

9. Project structure (high level)

Knowing where things live helps when you start making changes:

- **app/** — file-based routes (Expo Router). `_layout.tsx` wraps the tree in the context providers. `(tabs)/` holds the bottom-tab screens.
- **context/** — six providers, each persisting to AsyncStorage with a hasHydrated guard: ExerciseContext, TemplateContext, WorkoutContext, NutritionContext, UserProfileContext, and BodyMetricsContext.
- **data/exerciseCatalog.ts** — the static catalog of built-in exercises and the goal-based starter templates.
- **utils/** — helpers such as foodVision (Gemini), calorieBurn, nutritionGoals, and userApiKey.
- **components/ui/** — shared primitives like AppButton and AppCard.
- **assets/** — icons, splash screens, fonts.

10. Useful scripts

Defined in package.json:

```
npm run start          # alias for `expo start`
npm run ios            # `expo run:ios` (full native build)
npm run android        # `expo run:android`
npm run web            # `expo start --web`
npm run lint           # ESLint
npm run deploy:iphone  # Release build + install to a connected iPhone
```

Type-check the project without compiling:

```
npx tsc --noEmit
```

11. Troubleshooting

11.1 Metro is serving stale code

```
npx expo start -c
```

11.2 Weird native errors after pulling new code

```
rm -rf node_modules
npm install
cd ios && pod install && cd ..
```

11.3 iOS Simulator never opens

Open Xcode once, accept the license, and point the command-line tools at the full Xcode:

```
sudo xcode-select -s /Applications/Xcode.app/Contents/Developer
```

11.4 AI features say "Add your Gemini key"

No key is configured. Add one under Home → Profile → "AI features · Gemini key" (section 6), or set EXPO_PUBLIC_GOOGLE_AI_KEY in .env.local for local dev.

11.5 QR code scan fails on a real phone

- Confirm the phone and computer are on the same Wi-Fi.
- Some networks block the required ports; try a hotspot or tunnel mode: `npx expo start --tunnel`.
- If Expo Go cannot load a JS bundle, fully close Expo Go and rescan.

11.6 npm permission errors during install

Do not run npm with sudo. Instead, point the npm prefix at a folder you own:

```
mkdir -p ~/.npm-global
npm config set prefix '~/.npm-global'
export PATH=~/.npm-global/bin:$PATH   # add to ~/.zshrc
```

12. Pulling in updates

To grab the latest changes from GitHub:

```
git pull origin main
npm install        # in case new dependencies were added
```

If the Expo SDK version changes between pulls, also run:

```
npx expo install --check
```

13. Pushing your own changes

Make a feature branch, commit, and push:

```
git checkout -b my-feature
# ... make changes ...
git add .
git commit -m "Describe what you changed"
git push -u origin my-feature
```

Then open a pull request on GitHub to merge your branch into main.

14. Where to get help

- Expo documentation — docs.expo.dev
- React Native documentation — reactnative.dev
- Expo Router — docs.expo.dev/router/introduction
- Google AI Studio (Gemini keys) — aistudio.google.com
- Open an issue at github.com/gagangeet2517-arch/gymbro/issues for anything specific to this project.

End of manual. This PDF is generated from scripts/make_setup_pdf.py — edit that file and re-run it to update the guide.